

实验四：微信小程序开发

一、实验基本信息

项目	内容
实验名称	微信小程序开发——智慧校园通知系统
实验类型	验证型
开发工具	微信开发者工具、TRAE AI 编程助手
实验日期	2026 年

二、实验目的

1. 熟悉微信小程序的整体开发流程和项目架构
2. 掌握小程序页面结构设计和样式实现方法
3. 理解小程序页面间的导航与数据传递机制
4. 学习使用本地存储技术实现数据持久化
5. 体验智能辅助工具在开发过程中的应用
6. 完成一个功能完整的校园通知小程序

三、实验环境

- 操作系统：Windows 10
- 开发工具：微信开发者工具 Stable 版本
- AppID：微信小程序测试号
- AI 辅助工具：TRAE 智能编程助手
- 项目路径：d:\张天慈\移动应用开发\课堂数据文件\cg2mtcs\微信提醒

四、实验内容与实现步骤

4.1 需求分析与设计

页面流程设计

本次实验设计了一个包含通知列表和详情两个主要页面的小程序。用户可以在列表页面查看所有通知，并通过点击进入详情页面查看完整内容。系统会自动记录用户的阅读状态，确保用户可以清晰地分辨已读和未读的通知。

数据结构设计

为了实现通知管理功能，我们设计了包含通知标识、标题、内容、发布部门、发布时间和阅读状态等字段的数据结构，确保系统能够完整地存储和展示通知信息。

4.2 项目创建与配置

项目目录搭建

首先创建了标准的小程序项目目录结构，包括主目录和页面目录。主目录包含应用的入口文件和全局配置文件，页面目录则包含列表页和详情页的相关文件。

页面路由配置

在全局配置文件中设置了页面路由，确保小程序能够正确加载和切换不同页面。配置了列表页和详情页的路径，使系统能够在用户操作时进行正确的页面跳转。

4.3 通知列表页面实现

页面结构设计

列表页面采用卡片式布局，每条通知以独立卡片形式展示，包含标题、发布信息和未读标记。页面支持下拉刷新功能，方便用户获取最新通知。

核心功能实现

- 数据加载：**页面加载时从本地存储中读取通知数据，并更新每条通知的阅读状态。
- 页面导航：**点击通知卡片时，系统会跳转到详情页面，并传递当前通知的标识信息。
- 状态管理：**通过视觉效果区分已读和未读通知，提高用户体验。

4.4 详情页面实现

内容展示

详情页面展示通知的完整内容，包括标题、发布部门、发布时间和正文。页面布局清晰，确保用户能够舒适地阅读通知内容。

阅读状态处理

当用户进入详情页面时，系统会自动将当前通知标记为已读，并更新本地存储中的阅读状态记录。这样用户返回列表页面时，已读通知的状态会相应更新。

五、核心功能说明

5.1 本地存储技术

本地存储是实现数据持久化的关键技术，通过小程序提供的存储 API，可以将用户的阅读状态保存在本地。即使关闭小程序后重新打开，系统仍然能够保持之前的阅读状态，提供连贯的用户体验。



图一 本地存储实现已读标记

5.2 页面导航与数据传递

页面间的导航通过小程序的导航 API 实现，支持带参数的页面跳转。当用户点击通知卡片时，系统会将通知的唯一标识作为参数传递给详情页面，确保详情页面能够正确加载对应通知的内容。



图二 页面展示

六、实验遇到的问题与解决方法

问题描述	原因分析	解决方法
详情页返回后列表状态未更新	页面生命周期函数使用不当	利用页面显示时的生命周期函数，确保每次返回列表页时都刷新数据
通知内容换行显示异常	页面渲染机制限制	调整内容渲染方式，确保文本能够正确换行显示
未读标记位置偏移	样式布局问题	优化样式设置，确保未读标记在正确位置显示
首次运行时存储数据为空	初始状态处理不完善	为存储数据设置默认值，避免空值导致的异常

七、实验总结与心得体会

实验完成情况

✓ 全部功能已实现:

- 通知列表页面完整展示，布局美观
- 详情页面内容清晰，导航流畅
- 阅读状态管理功能正常，数据持久化有效
- 小程序在开发工具中运行稳定
- 支持下拉刷新等用户交互功能

收获与体会

1. **小程序开发特点：**小程序开发相比传统 Web 开发更加轻量，组件化程度高，开发效率提升明显
2. **数据驱动理念：**通过数据状态管理实现界面更新，理解了现代前端开发的核心思想
3. **本地存储应用：**掌握了客户端数据持久化的实现方法，提升了应用的用户体验
4. **智能工具价值：**智能辅助工具在开发过程中起到了重要作用，大幅提高了开发效率